

PORTFOLIO · AX PRODUCT DESIGN

차윤건

HR SaaS 프로젝트 디자이너

레퍼런스 체크를 시작으로 진단·분석 플랫폼까지
HR SaaS를 도메인으로 작업해오고 있습니다.

담당

프로덕트 디자인 · 리딩

도메인

HR SaaS · 인재 검증·채용

기간

2019 – 최근

01 서비스 통합 플랫폼 개발

02 진단 신뢰도 개선

03 제품 개발 워크플로우 AX 전환

OVERVIEW

포트폴리오 소개

레퍼런스 체크를 시작으로 진단·분석 플랫폼까지 HR SaaS를 도메인으로 작업해 오고 있습니다.

기업이 인재를 채용하고, 검증하고, 관리하는 과정에서 발생하는 데이터를 수집·분석해 더 나은 의사결정을 돕는 HR SaaS를 주로 다룹니다.

다만 그동안은 진단별로 제품이 분리되어 데이터가 서로 단절되어 있었습니다. 이 포트폴리오에서는 흩어져 있던 진단과 데이터를 하나의 경험으로 연결하고, 이를 확장해 온 과정을 풀어냅니다.

레퍼런스 체크

함께 일한 동료의 관점에서 후보자를 확인합니다.

성과·협업·신뢰도 등 이력서만으로 확인하기 어려운 정보를 입사 전 동료 평가를 통해 파악합니다.

다면 진단 · 360°

상사·동료·부하·본인의 시선으로 역량을 입체적으로 진단합니다.

수습 적합성과 재직자의 역량을 다양한 평가자의 관점에서 확인합니다.

조직문화 진단

구성원의 목소리로 조직의 건강도를 확인합니다.

심리적 안전감·협업·성장 환경 등을 데이터로 수집해 조직의 강점과 개선점을 파악합니다.

퇴직 리뷰

퇴직자의 경험에서 조직을 개선할 단서를 찾습니다.

퇴직 인터뷰를 통해 이탈의 배경과 원인을 파악하고, 반복되는 문제를 발견합니다.

통합 진단 플랫폼

통합 플랫폼

흩어진 진단과 데이터를 하나의 경험으로 연결합니다.

입사 전 레퍼런스 체크부터 수습·재직 중 진단까지 데이터를 연결하고, 서로 다른 시점의 결과와 조직문화 데이터를 교차 분석할 수 있도록 확장했습니다.

주요 경험

프로덕트 디자이너로서 초기 단계부터 제품을 리딩하며, 전화로만 이루어지던 레퍼런스 체크의 DX 전환부터 제품 확장과 플랫폼 통합, 최근의 AX 전환까지 제품이 성장하고 변화하는 전 과정을 함께했습니다.

2019 · DX 전환

프로젝트 참여

전화 중심으로 이루어지던 레퍼런스 체크를 웹 기반 서비스로 전환하는 과정에 참여했습니다.

2020-2024 · 제품 확장

제품 디자인 리딩

레퍼런스 체크를 시작으로 다면진단·조직문화·퇴직 리뷰로 제품을 확장하고, 입사 전부터 퇴사까지 이어지는 HR 라이프사이클로 연결했습니다.

최근 · AX 전환

제품 개발 방식 리딩

분리되어 있던 제품을 하나의 플랫폼으로 통합하고, AI 도구를 활용해 기획-디자인-개발 워크플로우를 재설계했습니다.

다뤄온 문제의 범위

레퍼런스 체크 DX 전환

전화로만 이뤄지던 레퍼런스 체크를 웹 서비스로 옮기며 B2B SaaS의 0→1을 만들었습니다.

코어 서비스로 비즈니스 모델 확장

진단 Core를 모듈화해 새로운 진단과 비즈니스 모델로 제품을 확장했습니다.

피드백 기반 제품 개선 루프

고객 만족도 리서치와 결과 분석·정기 피드백으로 평가 신뢰도를 검증하고 변별력을 높였습니다.

워크플로우 AX 전환

Figma·Cursor 등 AI 도구로 디자인-개발 워크플로우를 재구성해 제품 개발 리드타임을 단축했습니다.

PROJECT 01

서비스 통합 플랫폼 개발

진단 Core를 모듈화하고 데이터를 연계해 개별 서비스를 하나의 플랫폼으로 통합함으로써, 비즈니스 모델의 확장성을 높이고 유지보수 비용을 절감했습니다.

기획~런칭 전 과정 주도

비즈니스와 UX의 균형 설계

플랫폼 구조 설계

01 문제 정의

02 해결 전략

03 실험 및 디자인

04 결과 및 성과

문제 정의

기존 제품은 레퍼런스 체크와 개별 진단에 머물러 있었고, 제품 확장과 데이터 연결을 가로막는 세 가지 구조적 한계가 있었습니다.

01 단발성 제품 구조로 인한 확장의 한계

기존 제품은 레퍼런스 체크와 개별 진단에 머물러 있어, 한 인재의 입사 전부터 퇴사까지 이어지는 HR 라이프사이클로 제품을 확장하기 어려웠습니다.



문제 정의

02 데이터 단절로 인한 진단 신뢰성 검증의 부재

입사 전 레퍼런스 체크와 입사 후 실제 동료들이 평가한 수습 기간 다면진단 결과가 서로 단절되어 있었습니다.

때문에 “입사 전 평가가 실제 입사 후 평가와도 일치하는가?”, “지인을 통한 긍정적 평가가 결과에 영향을 미치지 않는가?”와 같은 질문을 검증하기 어려웠고, 진단 문항과 결과 해석의 신뢰도를 지속적으로 검증·고도화할 피드백 루프도 만들기 어려웠습니다.

● 입사 전 · 레퍼런스



● 입사 후 · 다면진단

두 데이터 단절 · 정합성 검증 불가

문제 정의

03 진단마다 반복되는 개발과 긴 출시 리드타임

질문지와 분석 보고서가 고정된 형태로 하드코딩되어 있어 새로운 진단이나 보고서 포맷을 추가할 때마다 개발이 필요했습니다.

그 결과 새로운 진단을 출시하는 데 6개월~1년의 개발 기간이 필요했고, 고객 역시 새로운 진단을 접할 때마다 별도의 학습과 적응이 필요했습니다.

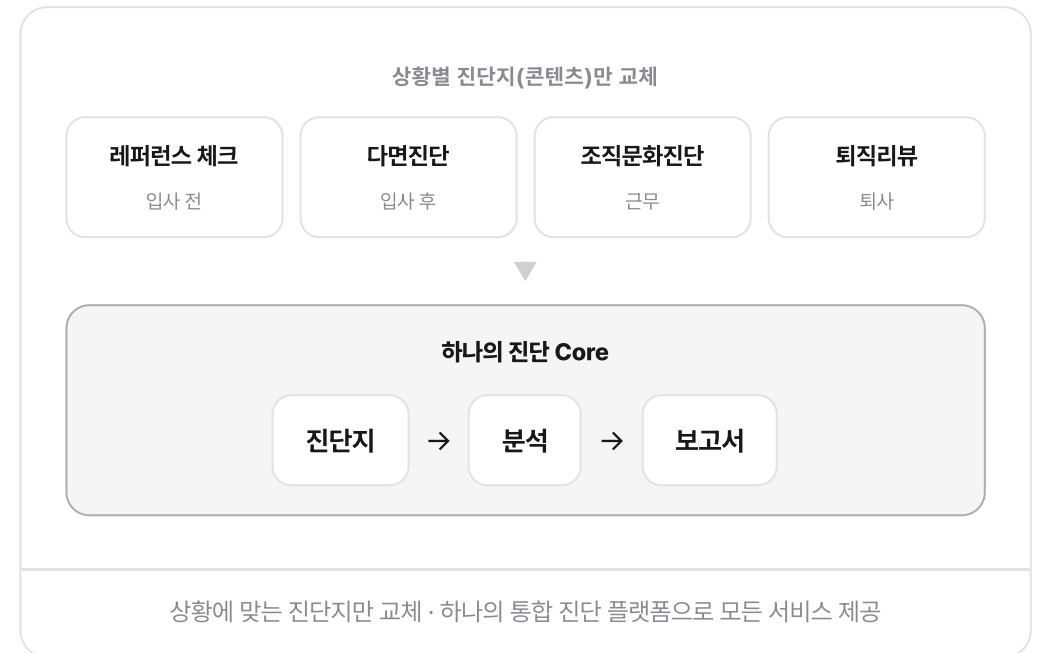
다면 진단		6-12개월
조직문화		6-12개월
퇴직 리뷰		~9개월

신규 진단마다 하드코딩 · 매번 6-12개월

해결 전략

01 진단 경험을 구성하는 Core 모듈화

진단지-분석-보고서로 이어지는 핵심 구조를 하나의 Core로 통합하고, 콘텐츠만 변경하면 새로운 진단을 확장할 수 있는 구조를 만들었습니다.



해결 전략

02 HR 라이프사이클 기반의 데이터 연결

입사 전 레퍼런스 체크부터 근무, 수습 종료 전 다면진단, 퇴사까지 이어지는 진단 구조를 만들고, 서로 다른 시점의 데이터를 연결해 사전 평가와 실제 업무 평가를 비교할 수 있는 구조를 만들었습니다.

03 PLG 기반 제품 확장

레퍼런스 체크 고객이 수습 종료 시점에 자연스럽게 다면진단까지 경험하도록 연결해, 영업 개입 없이 제품군이 확장되는 퍼널을 만들었습니다.

01 진단-분석-보고서의 공통 Core 시스템 구축

관리자가 별도의 개발 없이 진단 목적에 맞는 질문지를 직접 구성하고 프리셋으로 저장할 수 있도록 에디터 구조를 설계했습니다.

진단지, 분석, 보고서가 동일한 Core를 사용하도록 구조화해 새로운 진단을 접하더라도 기존과 동일한 UX를 경험하도록 했습니다.

이를 통해 진단마다 새롭게 학습해야 하는 부담을 줄이고, 하나의 UX 패턴 안에서 여러 진단을 사용할 수 있도록 설계했습니다.

진단지 에디터

질문 응답 통계 설정

저장 발행

레퍼런스 체크 · 표준

성과·협업·신뢰도를 함께 일한 동료에게 확인

함께 일한 기간

객관식(단일) ⌵

☐ 1년 미만 ×

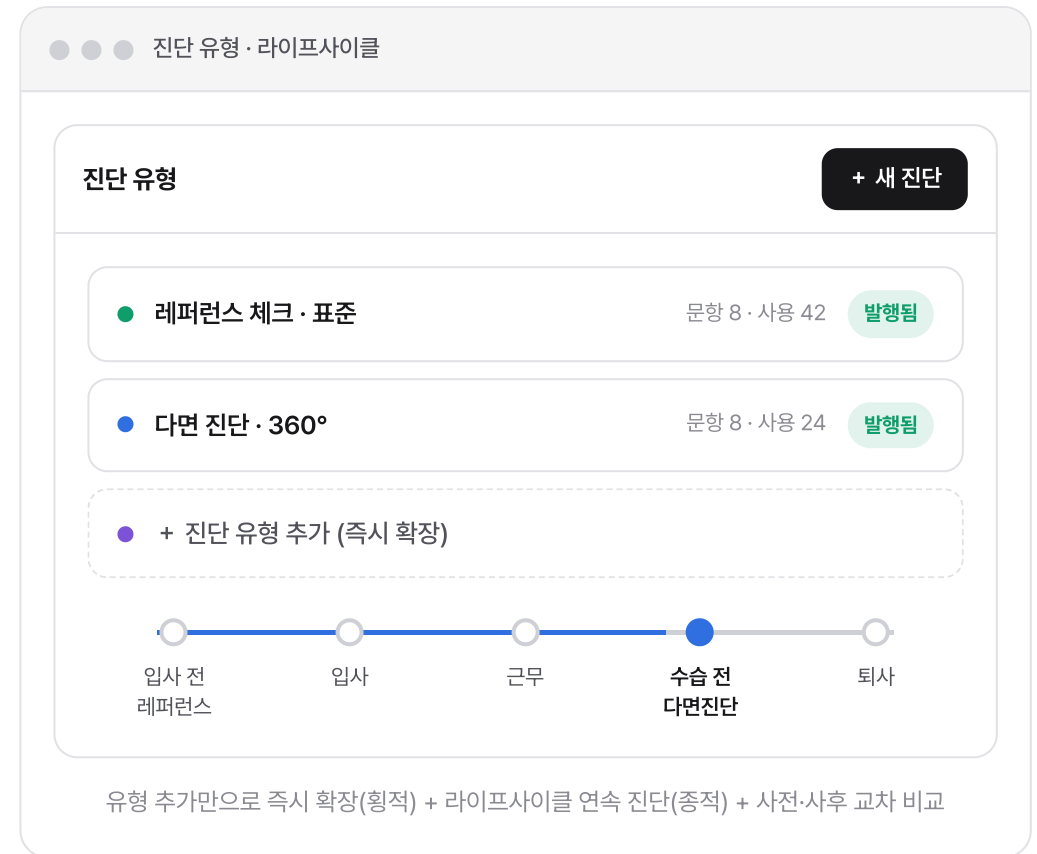
☐ 1~3년 ×

☐ + 옵션 추가

02 HR 라이프사이클 기반의 횡적·종적 확장

기존에는 새로운 진단을 하나씩 개발해야 했지만, 공통 Core 위에서 관리자가 새로운 진단을 구성할 수 있도록 해 콘텐츠를 바꾸는 방식으로 진단을 확장했습니다.

동시에 입사 전 레퍼런스 체크 → 근무 → 수습 종료 전 다면진단 → 퇴사로 이어지는 진단 흐름을 연결하고, 입사 전 레퍼런스 체크와 수습 종료 전 다면진단을 교차 비교할 수 있는 데이터 피드백 루프를 구축했습니다. 이를 통해 입사 전 평가가 실제 입사 후 평가와 얼마나 일치하는지 지속적으로 검증하고, 축적된 데이터를 기반으로 진단 문항과 평가 알고리즘을 개선할 수 있는 기반을 마련했습니다.

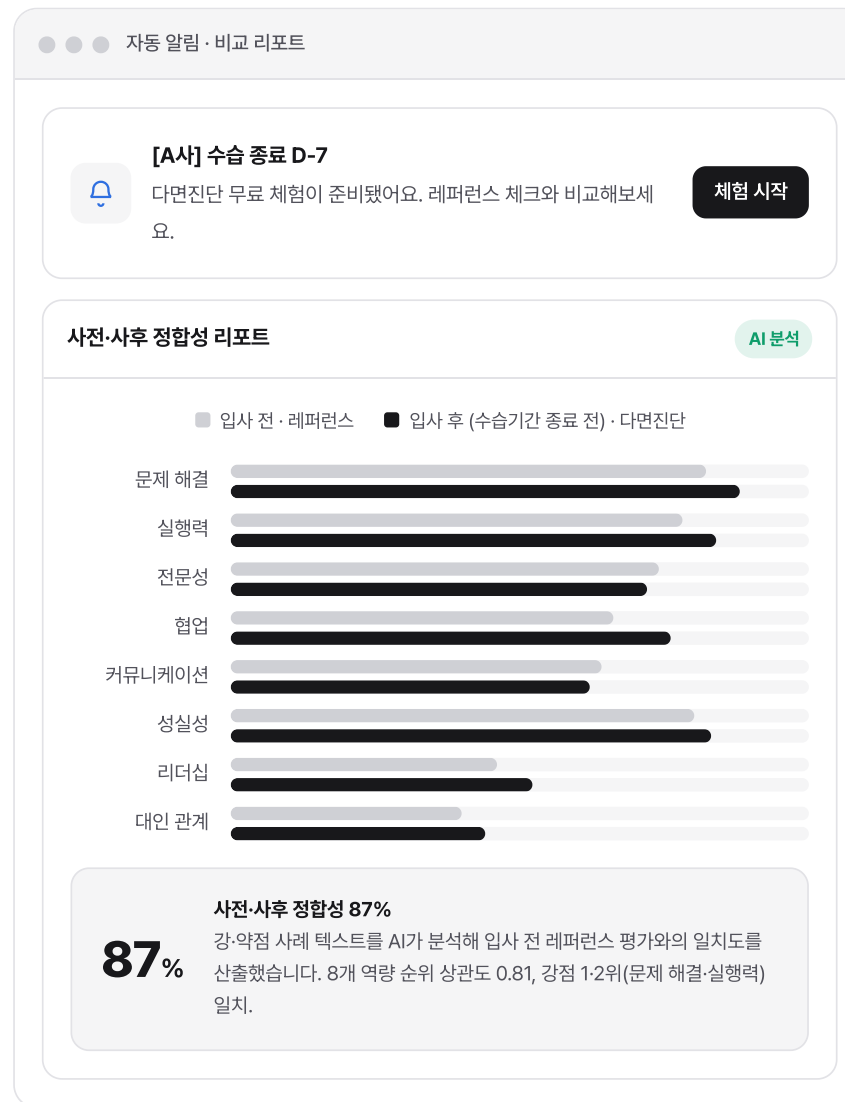


실험 및 디자인

03 PLG 기반 제품 확장 퍼널 설계

레퍼런스 체크만 이용하는 고객에게 수습 종료 시점에 자동 알림을 보내고, 다면진단을 무료로 경험할 수 있도록 설계했습니다.

두 진단의 결과를 비교한 리포트까지 제공해 기존 고객이 별도의 영업 접점 없이 다음 제품으로 확장할 수 있는 경험을 만들었습니다.



레퍼런스 체크 매출

400% ↑

DX 전환·플랫폼 확장 이후

신규 매출 파이프라인

대기업 실매출 확보

다면진단·조직문화진단 확장

신규 진단 리드타임

6-12개월 → 2-3개월

공통 Core 기반 확장

사전·사후 데이터

연결 구조 확보

교차 비교 기반 신뢰성 검증

결과 및 성과

공통 Core를 기반으로 신규 진단 출시 리드타임을 단축하고, 제품 간 데이터를 연결해 제품 확장과 진단 신뢰성 검증이 함께 가능한 구조를 구축했습니다.

결과 및 성과 · 세부 성과

01 레퍼런스 체크 매출 400% 성장

레퍼런스 체크를 웹 서비스로 전환하고 진단 제품군을 하나의 플랫폼으로 확장하면서, DX 전환 이후 매출이 400% 성장했습니다.

02 신규 매출 파이프라인 확보

레퍼런스 체크 고객이 수습 종료 시점에 다면진단·조직문화진단으로 자연스럽게 이어지도록 설계해, 영업 개입 없이 대기업 고객에서 신규 진단 제품의 실매출이 발생하는 파이프라인을 확보했습니다.

03 신규 진단 출시 리드타임 단축

질문지와 보고서가 하드코딩되어 6개월~1년이 걸리던 신규 진단 개발을 공통 Core 위에서 콘텐츠를 변경하는 구조로 재설계해 2~3개월로 단축했습니다.

04 데이터 신뢰성 검증 기반 확보

입사 전 레퍼런스 체크와 수습 종료 전 다면진단을 교차 비교할 수 있는 데이터 구조를 마련해, 진단 문항과 결과 해석의 신뢰도를 지속적으로 검증·고도화할 수 있는 기반을 확보했습니다.

PROJECT 02

진단 신뢰도 개선

점수 중심 평가를 순위 기반 평가로 전환해, 후보자의 역량별 강약을 한눈에 파악하고 채용 의사결정에 필요한 정보를 제공할 수 있도록 했습니다.

기획~런칭 전 과정 주도

데이터 기반 UX 설계·개선

사용자 중심 문제 정의

정량적 근거 기반

01 문제 정의

02 해결 전략

03 실험 및 디자인

04 결과 및 성과

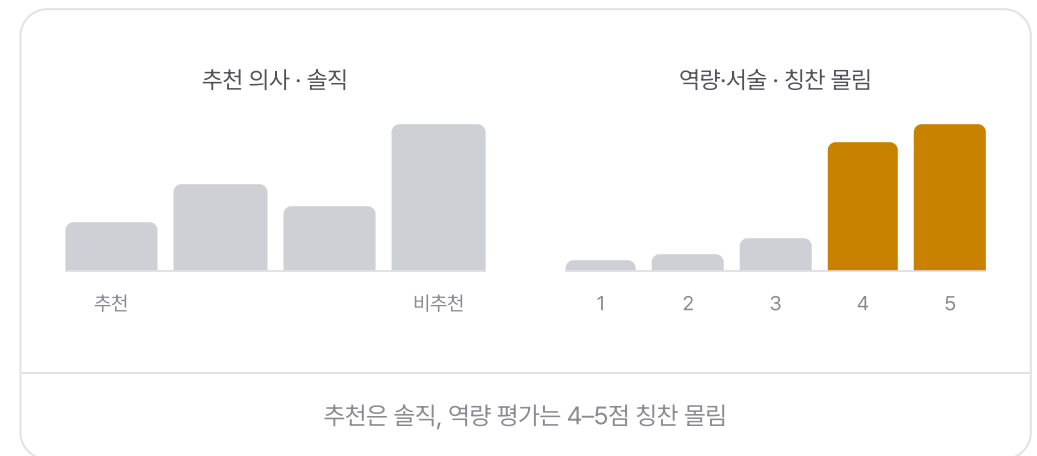
문제 정의

기존 역량 평가는 높은 점수에 응답이 몰리면서 후보자 간 차이를 구분하기 어려웠고, 추천 여부와 역량 평가의 목적도 하나의 흐름에 섞여 있었습니다.

01 높은 점수에 몰리는 응답

기존 5점 리커트 척도 기반 역량 평가에서는 응답자의 **99.1%가 4~5점**에 응답해 후보자의 강점과 약점을 구분하기 어려웠습니다.

반면 추천 여부에서는 약 10% 내외의 비추천 의견이 확인되어, 역량 평가와 추천 판단 사이에 데이터 변별력의 차이가 나타났습니다.

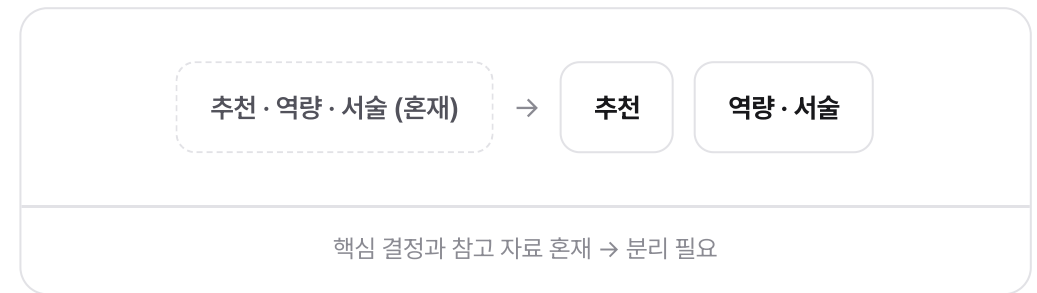


문제 정의

02 평가 목적이 하나의 흐름에 섞여 있었습니다

'추천 여부'는 채용 의사결정을 위한 핵심 판단인 반면, '역량·서술 평가'는 그 판단을 보완하는 참고 자료입니다.

하지만 두 평가가 하나의 흐름에 혼재되어 있어, 평가자가 역량 평가에서도 긍정적인 응답을 이어가기 쉬웠고 실제 채용 의사결정에 활용할 수 있는 정보의 밀도도 낮았습니다.



해결 전략

01 추천 여부와 역량 평가 분리

추천 여부를 평가의 최상단에서 먼저 판단하도록 하고, 이후 역량·서술 평가는 독립적인 참고 자료로 분리했습니다.

02 절대점수에서 상대적인 순위로 전환

5점 리커트 척도 대신 8개 역량의 상대적인 강점과 약점을 비교하는 Rank-order 방식을 도입했습니다.

03 순위와 실제 업무 사례 연결

상대적인 순위만 제공하는 데 그치지 않고, 평가자가 해당 순위를 부여한 구체적인 업무 사례까지 함께 수집해 결과의 맥락을 확보했습니다.

01 평가 흐름의 분리 — Decoupling

평가 시작 단계에서 추천 여부를 먼저 확정하도록 설계해 핵심 의사결정을 명확하게 분리했습니다.

이후 역량·서술 평가는 추천 여부와 분리된 참고 자료로 정의해, 추천 여부에 대한 심리적 부담 없이 후보자의 역량을 평가할 수 있도록 평가 흐름을 재구성했습니다.

추천 척도에는 BARS(행동 기반 평가 척도)를 적용해 '무조건 채용', '비슷한 유형의 후보자와 비교한 뒤 채용'처럼 구체적인 행동 기준을 제시했습니다.

● ● ● 평가자 화면 · 추천 확정

이 후보를 추천하시겠습니까? BARS · 행동 기반 5점 척도

☐ 무조건 채용하겠습니다

☒ 비슷한 유형의 후보자와 비교한 뒤 채용하겠습니다

☐ 다른 대안이 없다면 채용하겠습니다

☐ 채용을 권하지 않습니다

☐ 어떤 경우에도 채용하지 않겠습니다

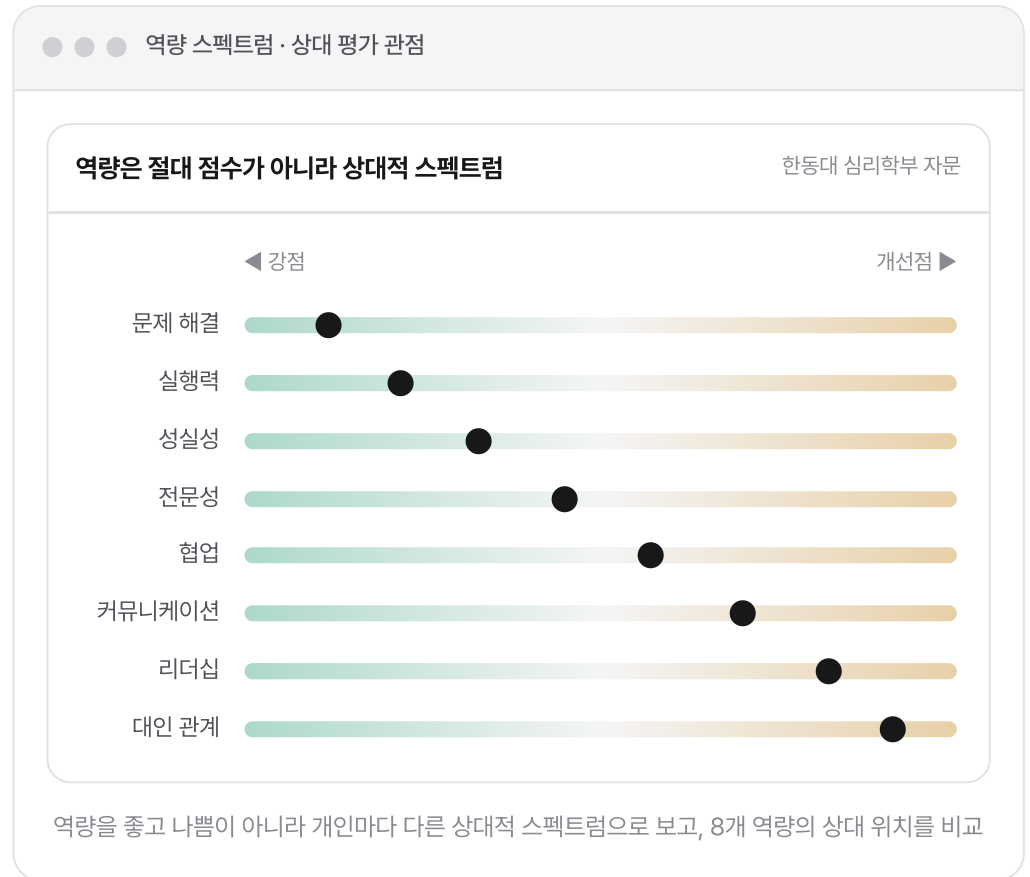
BARS 행동 기준으로 추천 여부를 먼저 확정, 역량·서술 평가는 참고 자료로 분리

02 평가 방법론 검토 — Rank-order

리커트 척도를 대체할 방법을 검토하면서 BARS를 역량 평가에도 적용하는 방안을 함께 검토했습니다.

한동대학교 심리학부와 역량검사 문항을 개발하며 평가 방법론에 대한 자문을 받고, 역량을 단순한 우열로 평가하기보다 개인마다 강점과 약점이 서로 다른 스펙트럼으로 존재한다는 관점을 채택했습니다.

이에 따라 8개 역량을 절대적인 점수로 평가하기보다 상대적인 위치를 비교하는 Rank-order 방식을 도입했습니다.



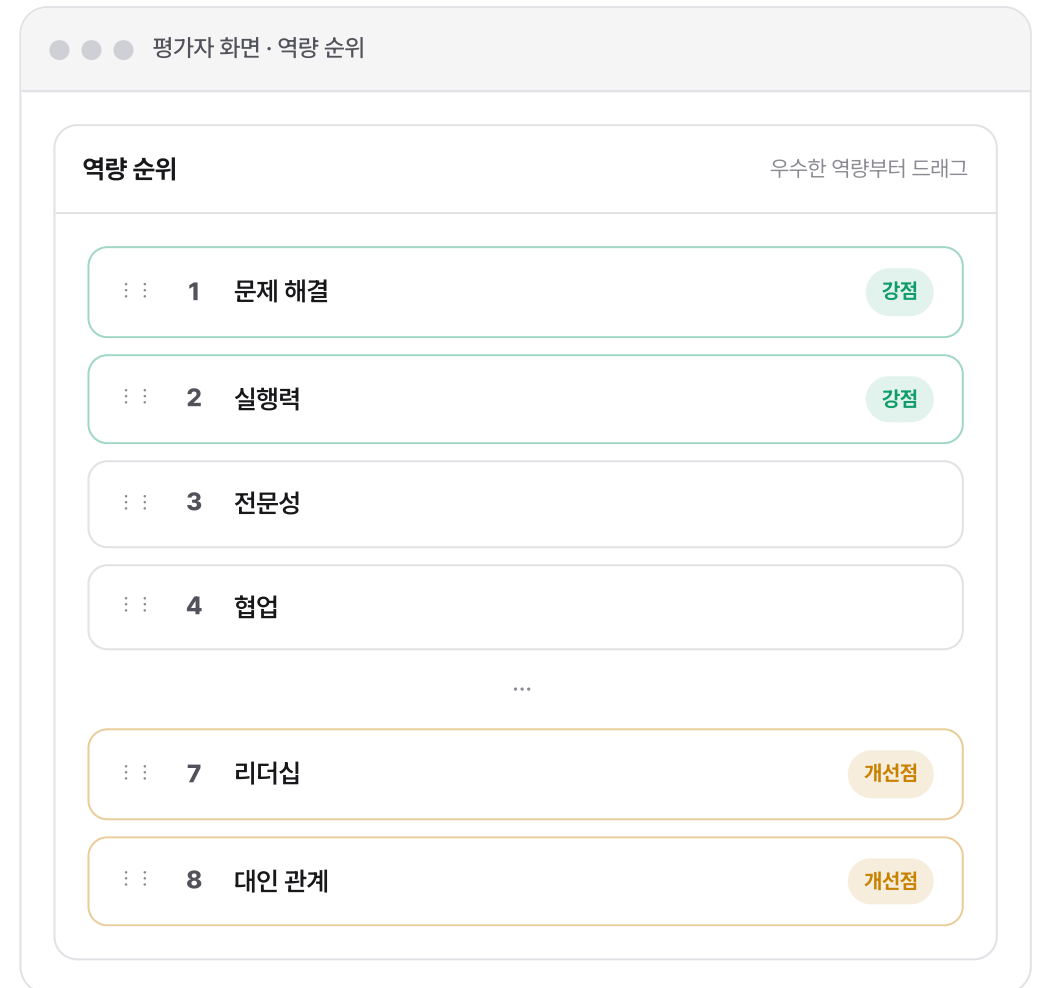
03 인터랙션 검증 — Dropdown → Drag & Drop

초기에는 각 역량마다 순위를 선택하는 Dropdown 방식으로 설계했습니다.

내부 직원 테스트에서 역량의 순서를 정하는 과정에서 마지막 몇 개의 카드를 두고 반복적으로 고민하며 순서를 바꾸는 행동이 관찰됐습니다.

기존 방식에서는 순위를 변경할 때 각 항목을 다시 선택해야 했기 때문에, 전체 순서를 한눈에 확인하고 즉시 재조정할 수 있도록 Drag & Drop 방식으로 변경했습니다.

사용자가 고민하는 과정에서 카드를 직접 움직이며 전체 순서를 확인하고, 필요한 순간 바로 순위를 변경할 수 있도록 인터랙션을 설계했습니다.



04 순위와 실제 사례 연결 — Contextual Linking

상위 2개 역량은 강점, 하위 2개 역량은 개선점으로 정의하고 해당 역량에 순위를 부여한 구체적인 업무 사례를 작성하도록 설계했습니다.

순위만으로 끝나지 않도록 왜 해당 역량을 상·하위로 평가했는지에 대한 실제 업무 맥락까지 함께 수집했습니다.

이를 통해 상대적인 순위 데이터와 정성적인 사례를 연결해 실제 채용 의사결정에 활용할 수 있는 정보를 제공했습니다.

● ● ● 평가자 화면 · 사례 기술

강점 · 1위

문제 해결

이 역량을 1위로 꼽은 구체적 사례를 적어주세요...

개선점 · 8위

대인 관계

이 역량을 마지막 순위로 둔 구체적 사례를 적어주세요...

결과 및 성과

점수 몰림으로 변별력이 없던 평가를 순위 기반 평가와 사례 연계 방식으로 재설계해, 후보자의 상대적인 강점·약점과 이를 뒷받침하는 정성적 근거를 함께 확인할 수 있는 평가 데이터로 전환했습니다.

대기업 고객사 만족도

60점대 → 85점대

헤비유저 3곳 직접 방문·인터뷰

평가 데이터 변별력

확보

8개 역량 상대 순위 전환

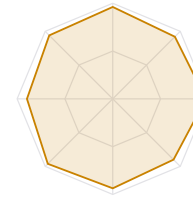
사례 기반 정성 데이터

확보

강점·개선점 사례 연결

● ● ● 8역량 결과 · 기존 그래프 vs 실질 데이터

기존 · 방사형 그래프



99.1%가 4-5점 → 변별력 없음

개선 · 순위 기반 + 실제 사례 연계

강점

1 문제 해결

1순위

사례 “결제 장애 시 원인을 빠르게 좁혀 12시간 내 복구를 주도했다.”

2 실행력

2순위

사례 “분기 목표를 2주 앞당겨 완료해 팀 리듬을 끌어올렸다.”

개선점

7 리더십

7순위

사례 “방향을 먼저 제시하기보다 합의를 기다리는 편이었다.”

8 대인 관계

8순위

사례 “의견 충돌 시 대화를 미뤄 조율에 시간이 걸렸다.”

무의미한 그래프 해석이 아니라, 강점·개선점을 선명하게 드러내고 실제 사례까지 연계된 실질 데이터를 제공

결과 및 성과 · 세부 성과

01 대기업 고객사 만족도 상승

평가 방식에 컴플레인이 많던 헤비유저 대기업 고객사 중 3곳을 무작위로 선별해 개선안을 적용하고, 고객사를 직접 방문해 인터뷰했습니다.

그 결과 만족도 점수가 60점대에서 85점대로 상승했습니다.

02 평가 데이터의 변별력 확보

응답자의 99.1%가 4~5점에 몰렸던 기존 리커트 평가를 8개 역량의 순위 기반 평가로 전환해, 역량 간 상대적인 차이가 데이터로 드러나도록 개선했습니다.

03 사례와 연결된 실질 데이터 확보

상·하위 역량에 구체적인 업무 사례를 연결해, 단순 점수나 그래프로는 파악하기 어려운 후보자의 강점·약점과 실제 업무 맥락을 함께 확인할 수 있도록 했습니다.

PROJECT 03

제품 개발 워크플로우 AX 전환

AI 기반 워크플로우를 구축해 디자인과 개발 사이의 반복 작업을 줄이고, 스프린트 주기를 단축해 제품 개발 속도를 높였습니다.

기획~런칭 전 과정 주도

디자인 시스템 구축·운영

Figma·AI 협업 도구 활용

직군 간 협업 리드

01 문제 정의

02 해결 전략

03 실험 및 디자인

04 결과 및 성과

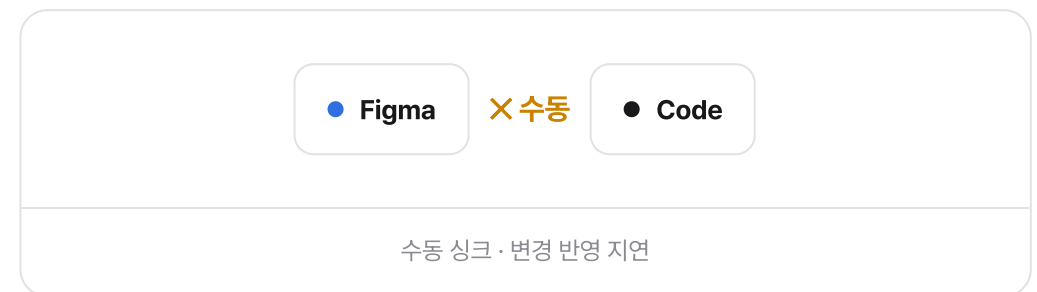
문제 정의

빠르게 제품을 고도화하는 과정에서 디자인 시스템과 코드 사이의 반복적인 싱크 작업, 그리고 신규 기능을 검증하기 위한 0→1 프로토타이핑의 반복이 제품 개발 속도를 떨어뜨리고 있었습니다.

01 디자인 시스템과 코드 사이의 반복적인 싱크 작업

서비스가 지속적으로 고도화되면서 디자인 시스템 업데이트와 UI 컴포넌트의 변경 사항을 수동으로 관리하는 작업이 반복됐습니다.

변경 사항을 개발에 전달하고 다시 확인하는 과정에서 디자인-개발 간 커뮤니케이션 비용과 개발 지연이 발생했습니다.

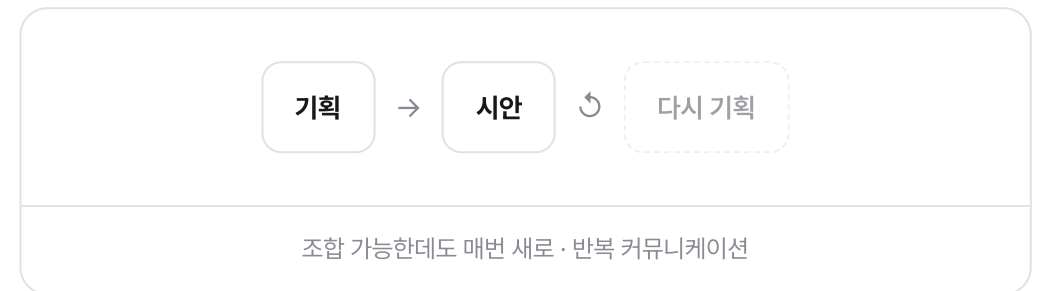


문제 정의

02 0→1 프로토타이핑의 반복

신규 기능을 개발할 때 기존 컴포넌트의 조합만으로 구현할 수 있는 경우에도 매번 기획과 시안을 새롭게 구성해야 했습니다.

그 결과 아이디어를 실제 제품 형태로 검증하기까지 시간이 오래 걸렸고, 디자인-개발 사이의 불필요한 커뮤니케이션도 반복됐습니다.



해결 전략

01 디자인 정보를 코드 생성 과정에 연결

Figma의 디자인 정보와 컴포넌트 구조를 AI가 활용할 수 있도록 연결해 디자인과 코드 사이의 반복적인 전달 작업을 줄였습니다.

02 AI 기반 프로토타이핑

디자이너가 직접 아이디어를 구현하고 검증할 수 있는 환경을 구축해 기획-디자인-개발 사이의 간극을 줄였습니다.

03 디자인 시스템과 Spec 구조화

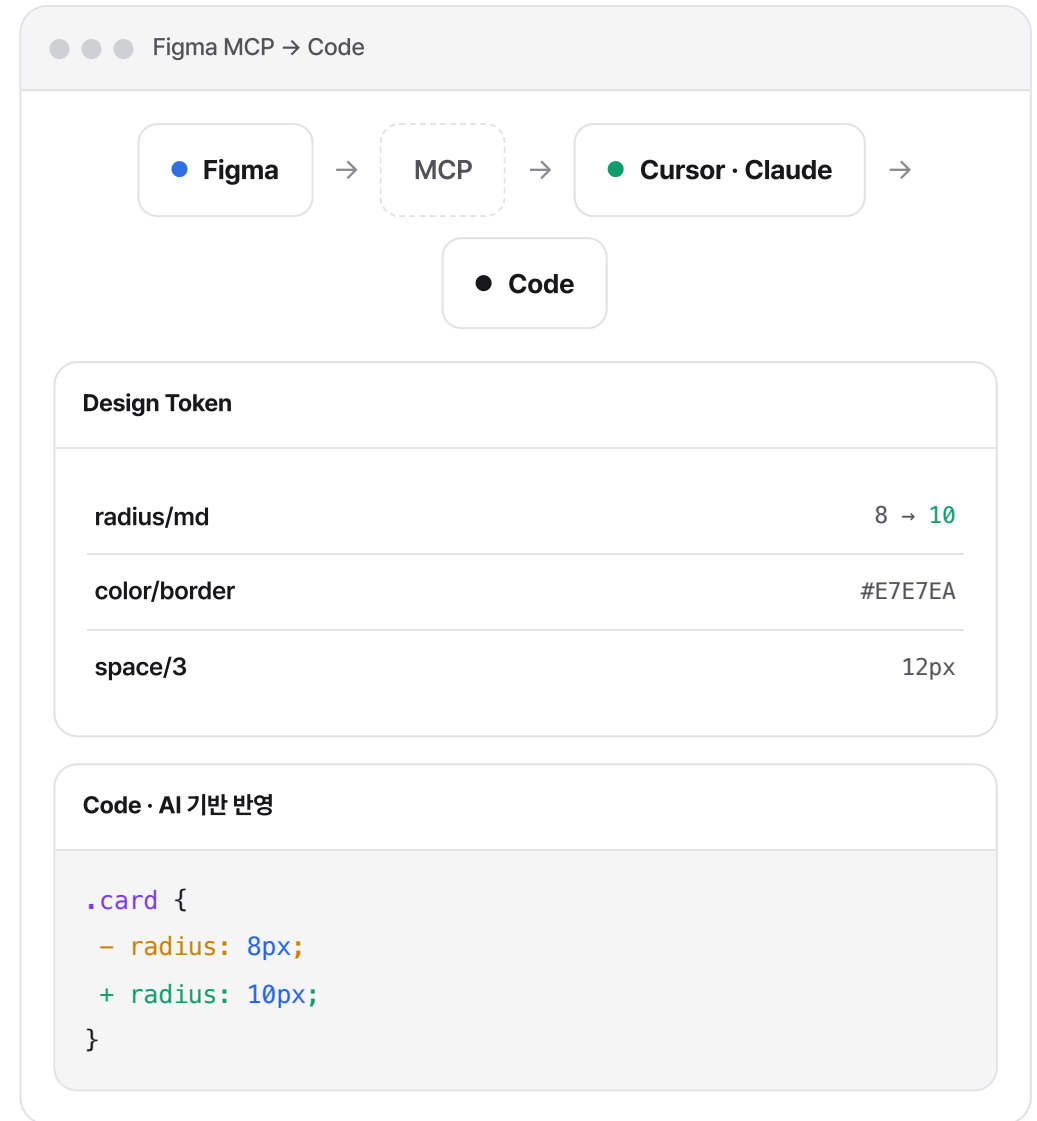
컴포넌트와 디자인 토큰, 상태값을 구조화해 AI와 개발자가 동일한 정보를 활용할 수 있도록 하고, 반복적인 문서 작성과 핸드오프 비용을 줄였습니다.

실험 및 디자인

01 Figma MCP · 디자인-코드 연결 워크플로우

Figma 환경과 LLM(Claude/Cursor)을 MCP(Model Context Protocol)로 연결해 디자인 시스템의 구조와 컴포넌트 정보를 AI가 활용할 수 있는 워크플로우를 구축했습니다.

디자인 정보를 코드 생성·수정 과정에 연결해 디자인과 구현 사이의 불일치를 줄이고, 변경 사항을 빠르게 반영할 수 있는 기반을 마련했습니다.

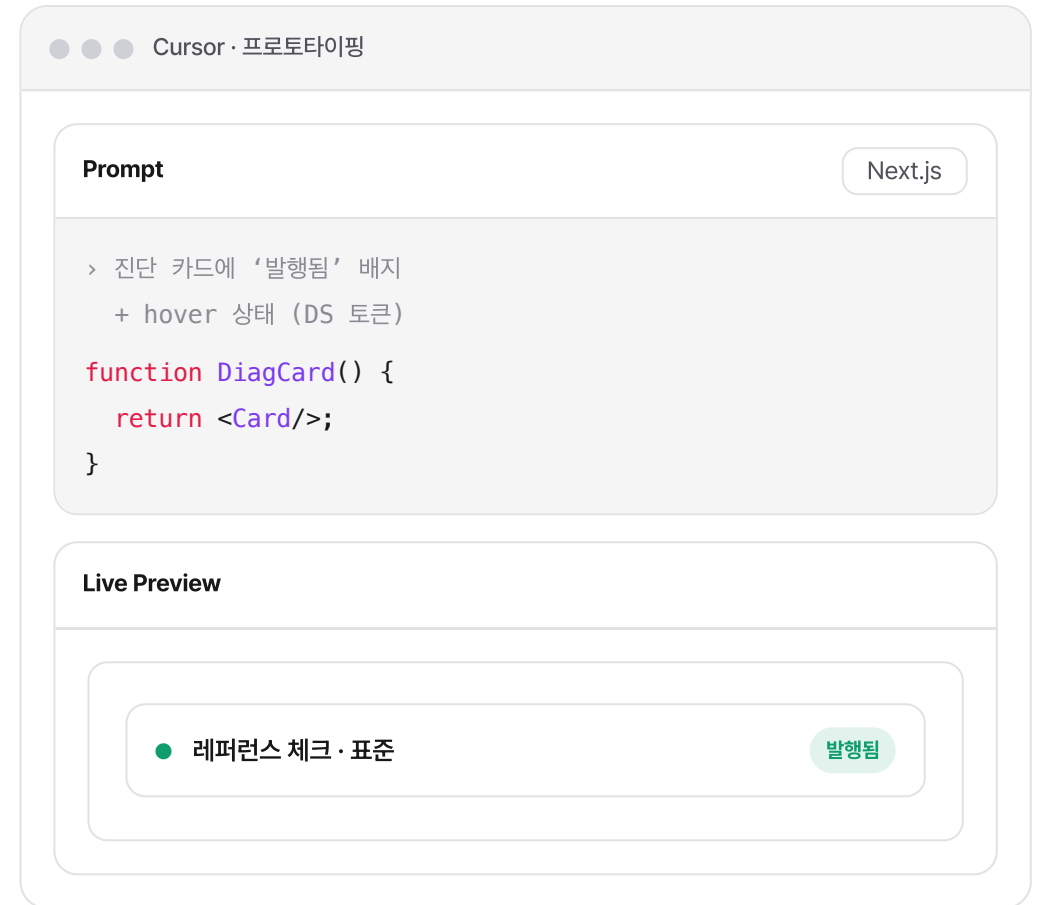


실험 및 디자인

02 AI 기반 프로토타이핑을 통한 가설 검증 가속화

검증된 기술 스택인 Next.js / Supabase / Vercel 환경에서 Cursor를 활용해 디자이너가 아이디어를 직접 프론트엔드 프로토타입으로 구현하고 제품 스펙을 구체화했습니다.

기획 → 디자인 → 구현의 간극을 줄이고, 실제 사용 환경에서 검증할 수 있는 단계까지 빠르게 도달하는 제품 검증 루프를 구축했습니다.



03 디자인 시스템 문서화 · Spec 정의 간소화

UI 컴포넌트의 상태값과 디자인 토큰을 AI가 활용할 수 있도록 구조화하고, 이를 기반으로 개발자에게 전달되는 기능 명세서(Spec) 작성과 핸드오프 과정을 간소화했습니다.

반복적인 문서 작성과 커뮤니케이션에 드는 시간을 줄이고, 디자이너가 UX 의사결정과 제품 검증에 더 집중할 수 있는 환경을 만들었습니다.

Auto-generated Spec	
Component States · Card	
Default	bg surface · border 1px
Hover	border → foreground/25
Active	bg foreground/5
Disabled	opacity 0.5 · no-pointer

제품 출시 리드타임

6개월 → 2개월

기획/개발 각 1개월 스프린트 병렬

디자인-개발 소통 공수

90% ↓

미팅 시간 기준

디자인 시스템

일관된 관리 체계

반복 작업 감소

결과 및 성과

반복적인 디자인-개발 커뮤니케이션과 수작업을 줄이고, 디자이너가 아이디어를 직접 구현하고 검증할 수 있는 AI 기반 제품 개발 파이프라인을 구축했습니다.

결과 및 성과 · 세부 성과

01 제품 출시 리드타임 6개월 → 2개월

기획과 개발을 각각 1개월 스프린트로 병렬화하고 AI 프로토타이핑을 통해 검증을 앞당겨, 6개월이 걸리던 제품 출시를 2개월로 단축했습니다.

02 디자인-개발 소통 공수 90% 절감

Figma 기반의 디자인 정보를 AI와 코드 생성 과정에 연결하고, 컴포넌트 상태와 Spec 전달 방식을 구조화해 미팅 시간 기준 디자인-개발 소통 공수를 90% 줄였습니다.

03 디자인 시스템 상시 관리 체계 구축

수동으로 반복되던 디자인 시스템 관리와 전달 과정을 구조화해, 변경 사항을 빠르게 반영하고 UI 일관성을 지속적으로 유지할 수 있는 관리 체계를 구축했습니다.

디자이너 역할의 확장

기획 → 디자인 → 개발 → 검증이 분리된 구조에서는 디자인이 끝난 뒤 구현 단계에서 발생하는 차이를 다시 맞추는 핸드오프가 반복되었습니다. 기존 컴포넌트를 조합하는 간단한 기능조차 매년 디자인-개발 사이의 커뮤니케이션을 거쳐야 했습니다.

이를 개선하기 위해 Figma와 AI 개발 환경을 연결하고, 디자이너가 Cursor를 활용해 직접 프로토타입을 구현하고 검증하는 워크플로우를 구축했습니다. 아이디어를 빠르게 실제 동작하는 화면으로 만들고, 직접 테스트하며 스펙을 구체화한 뒤 개발로 전달하는 방식입니다.

그 결과 디자이너의 역할을 화면을 설계하고 전달하는 것에서, 직접 구현하고 검증하며 제품 완성도를 높이는 것까지 확장했습니다. 디자인과 개발 사이의 경계를 줄이고, 아이디어 → 구현 → 검증의 사이클을 짧게 반복하는 제품 개발 방식을 만들었습니다.

AI는 디자인을 대체하는 도구가 아니라 디자인과 개발 사이의 거리를 줄이는 도구로 활용했습니다. 디자이너는 구현 가능성을 더 빠르게 검증하고, 개발자는 완성된 시안이 아니라 실제 동작하는 프로토타입을 기반으로 구현하도록 협업 방식을 바꿨습니다.

